



ECE 370 Lab I

Dongjun Lee

School of Electrical Engineering and Computer Science
Oregon State University



Lab Information

- All lab materials can be found on Canvas:
 - Lab Handouts and PPT slides for the lab instructions.
 - Skeleton Codes (and Example Codes, if available).
 - Simulator Installation Guides.
 - Datasheets and AVR Instruction Set Manual.
 - Report templates.
- Assignments should be submitted via Canvas.
- Gradings for each lab will be updated by the beginning of the following week's lab.



Must-know Policies

- Lab sessions:
 - Will be used for check-offs.
 - Only your lab TA can get you checked off.
 - You must be present for check-offs.
- Office Hours:
 - TAs will hold their office hours in the lab.
 - Asking for debug-helps must be done during office hours or separate appointments.
 - Note that we do not respond to your email that contains code because we need to check both hardware and software.

Work Policy (I)

- You can work alone or in a group:
 - A group can have **no more than two people**.
 - Your partner should belong to **the same lab session**.
 - You can change your partner for each lab.
- Submission:
 - Each individual must submit lab assignments via Canvas.
 - Must **include your (and partner's) names** to prevent plagiarism issues in
 - **Cover page** in the **lab report**
 - **Comments** in the beginning of the **CODE**.
 - **File name** of the **CODE** and **report files**.
 - Lab reports must be done **independently**.
 - Lab reports must be done using **templates** in the Lab Material section.
 - You and your partner must submit the exact **same code file**.



Work Policy (2)

- No late work is allowed:
 - 2 weeks labs: Lab 1 and Lab 7.
 - 1 week labs: Lab 2 – 6.
- Recycling your work from the previous terms is **forbidden** and will be flagged as plagiarism.



Plagiarism Policy

- You are never allowed to represent another person's work as your own even if it is just a small part.
- If you plagiarize, **you will be reported to the school/college for academic dishonesty.**
- If you are in a group, both you and your partner are responsible.
- Plagiarism checking will happen at a random time:
 - Don't waste your entire term's work by falling for a temptation.



Plagiarism Includes

- Excluding your partner's name:
 - The plagiarism checker will raise a red flag when you both submit the same file but does not specify both names.
- Recycling your code from previous terms:
 - You can't reuse the previous work of yours or your partner's.
- Partially picking or copying code:
 - Cherry picking for code is a bad temptation.
 - It doesn't matter whether you understand the code or not.
- Deceiving by changing names of registers or comments.
 - Don't presume the plagiarism checker is stupid enough.



Gratings

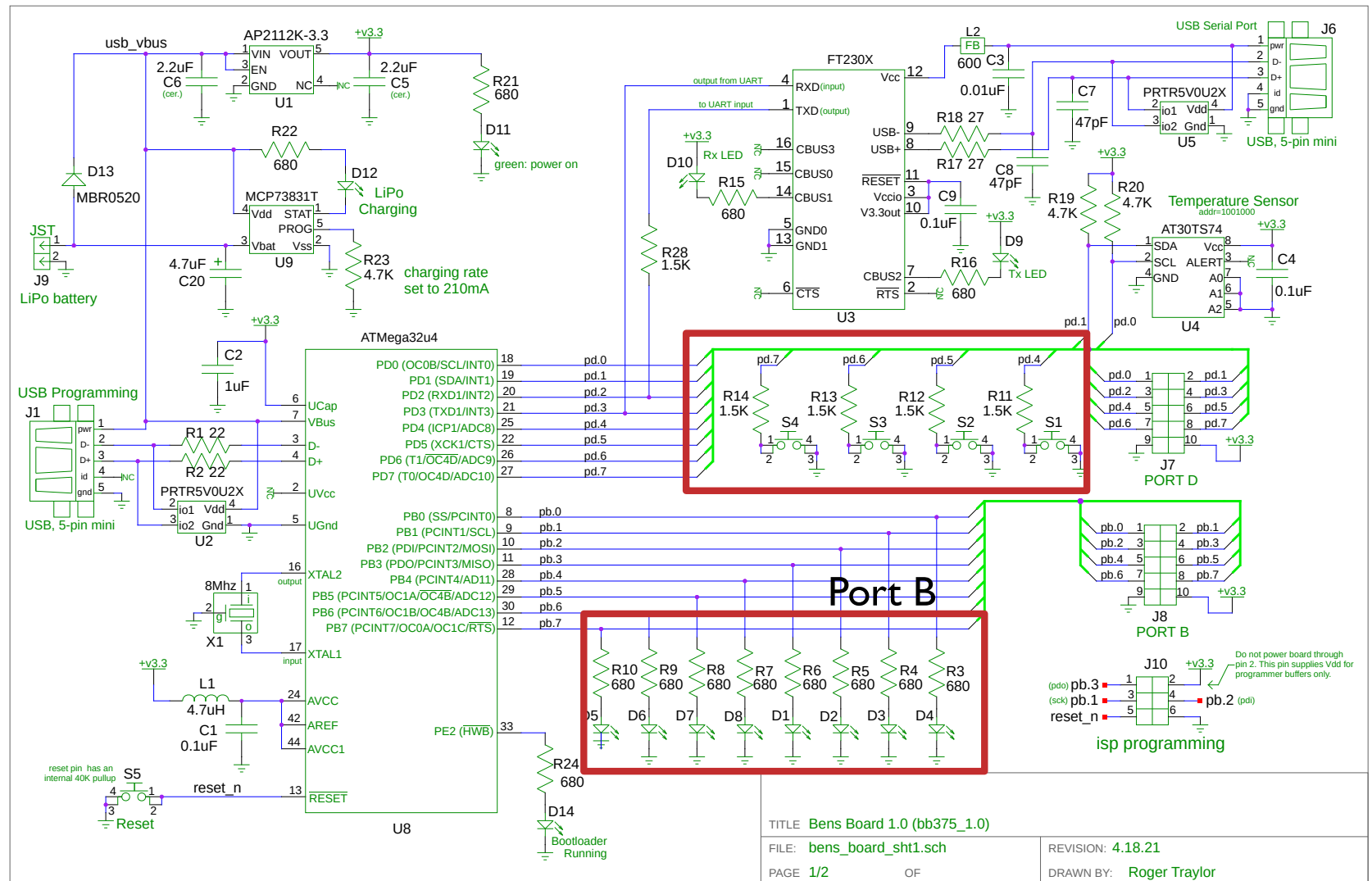
- Lab reports: 30%
- Check-offs: 70%
- You will receive zero for both if you miss either a lab report or check-off.

Lab I Introduction

- Lab I will familiarize you with software and the board.
 - Follow the **Installation Guides** in the Lab I Materials.
- Part 1 (Week 1)
 - Download the `BasicBumpBot.asm` code and program into your AVR board.
 - **Figure out** how TekBot **reverses for twice as long** before turning away and resuming forward motion. Provide your answer in Study Question 4.
- Part 2 (Week 2)
 - Download the `DanceBot.c` code and understand how to configure the I/O ports.
 - Write a simple C program to replicate the BumpBot behavior in Part 1.
- Attend your checkoff session and demonstrate your work.

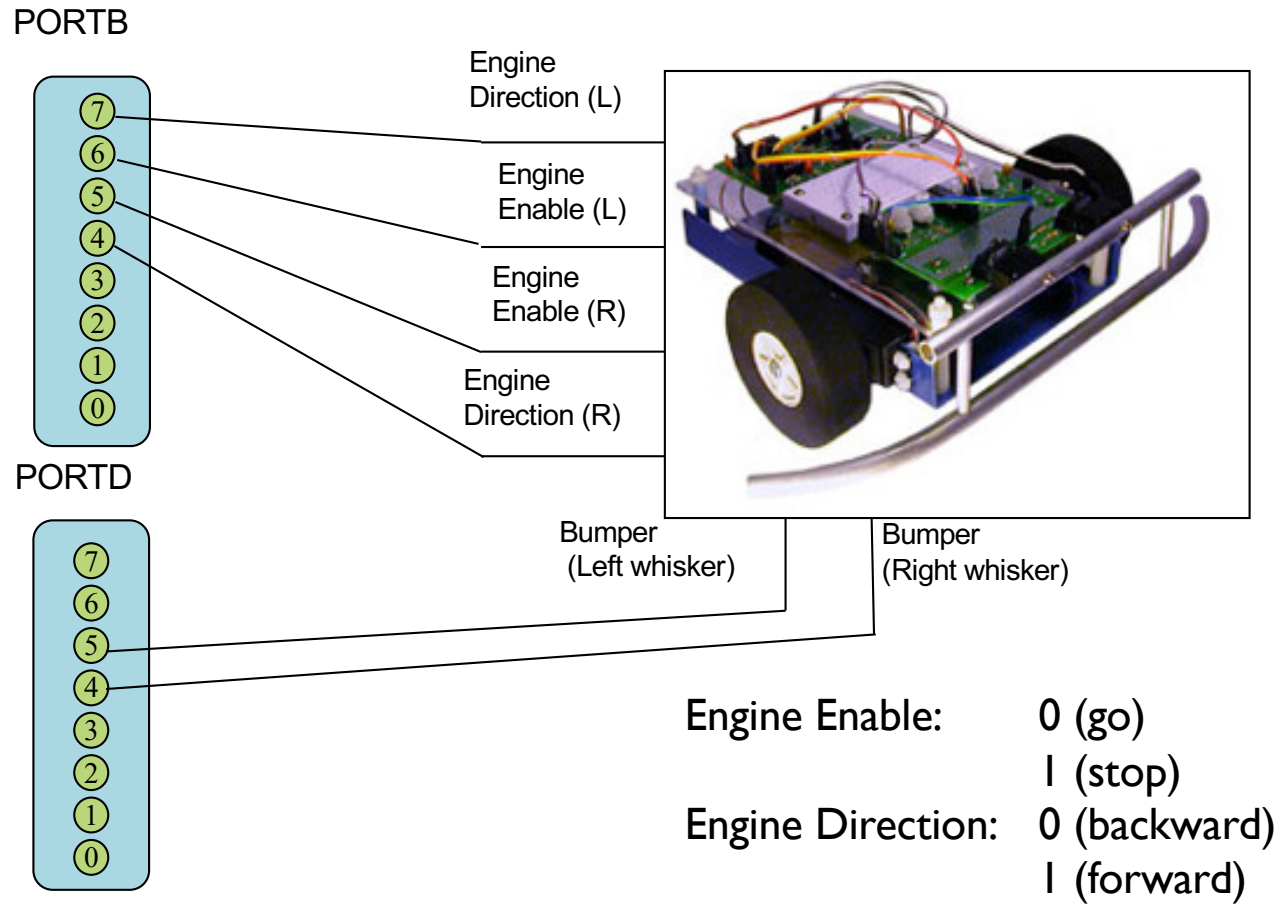
Lab I Check-off Instructions

- Download your submitted C code in Canvas in front of your Lab TA.
- Compile and flash the code to an AVR board.
- Demonstrate that it is working correctly:
 - Even if you did not complete the lab, you still need to show your work to the TA to get partial credits.
- Explain your code:
 - TA will ask some questions regarding the code. Some credits will be taken away if you are unable to answer these questions.
 - You are required to add a comment to every line of your code.



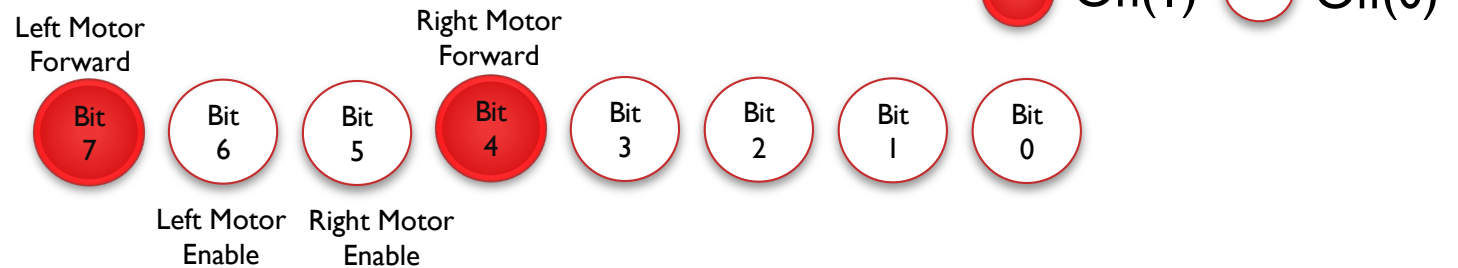
Connection Guides

Bumper hit closes a switch causing high-to-low transition.

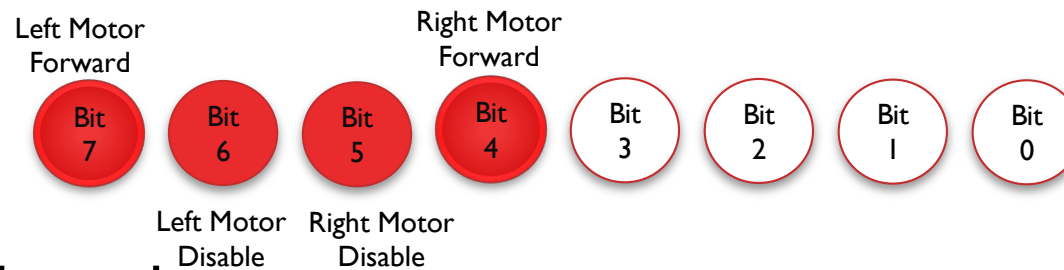


Bumpbot Behavior (I)

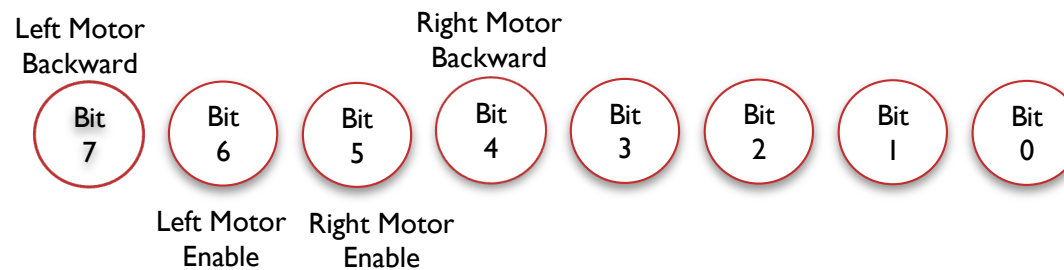
- Forward



- Halt



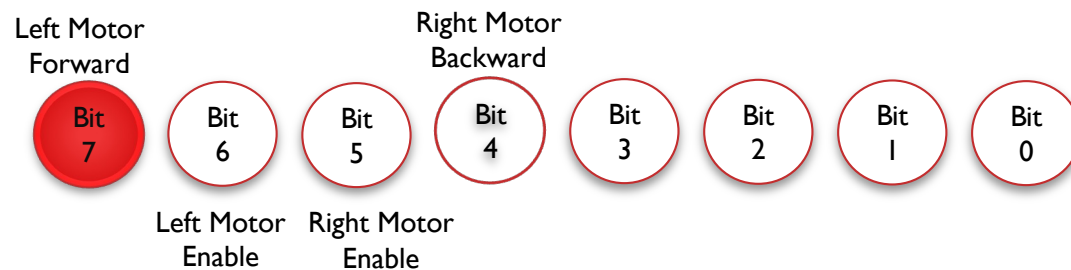
- Backward



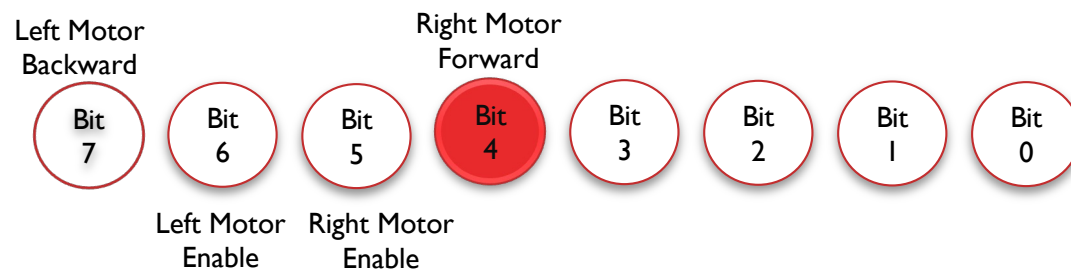
Bumpbot Behavior (2)

- Turn Right

 On(1)  Off(0)



- Turn Left



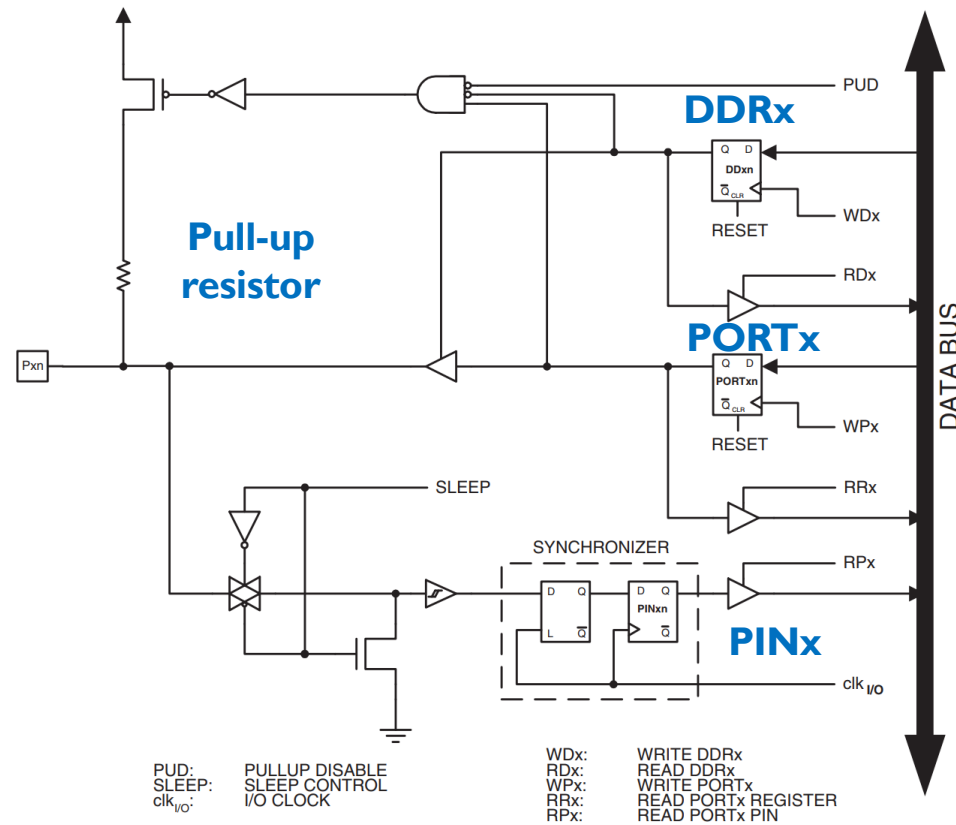
Control Registers

- Three types of registers:
 - DDRx is a Data Direction Register for Port x
 - PORTx is a Port Output RegisTer for Port x
 - PINx is a Port Input register for Port x
- Output Port Settings:
 - DDRB = 0b11111111 ; Set bits 7-0 as outputs
 - PORTB = 0b11110000 ; Turn on LEDs connected to bits 7-4 bits
- Input Port Settings:
 - DDRD = 0b00000000 ; Set bits 7-0 as inputs
 - PORTD = 0b11111111 ; Enable pull up resistor
 - IN mpr, PIND ; Read input data to mpr

AVR Ports

Atmega32U4 Datasheet p. 67

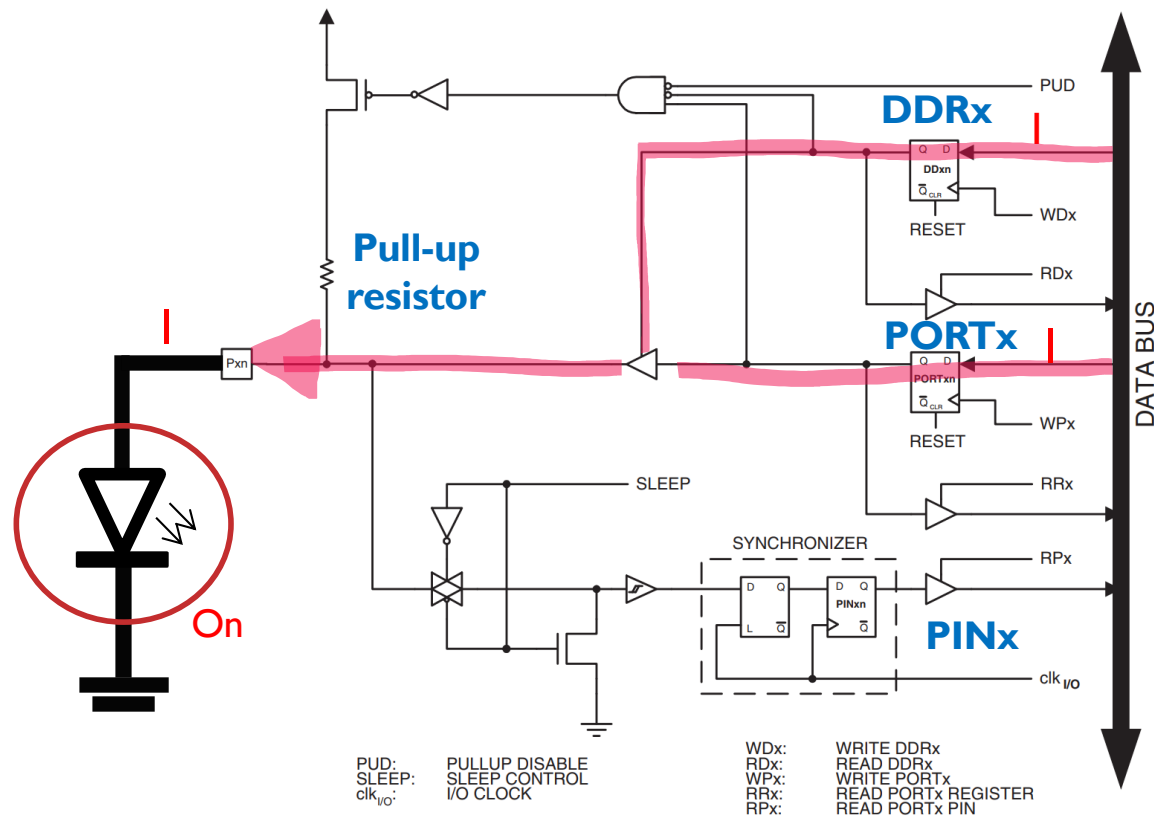
Figure 30. General Digital I/O⁽¹⁾



Note: 1. WP_x , WD_x , RR_x , RP_x , and RD_x are common to all pins within the same port. $clk_{I/O}$, $SLEEP$, and PUD are common to all ports.

Configure for Output (I)

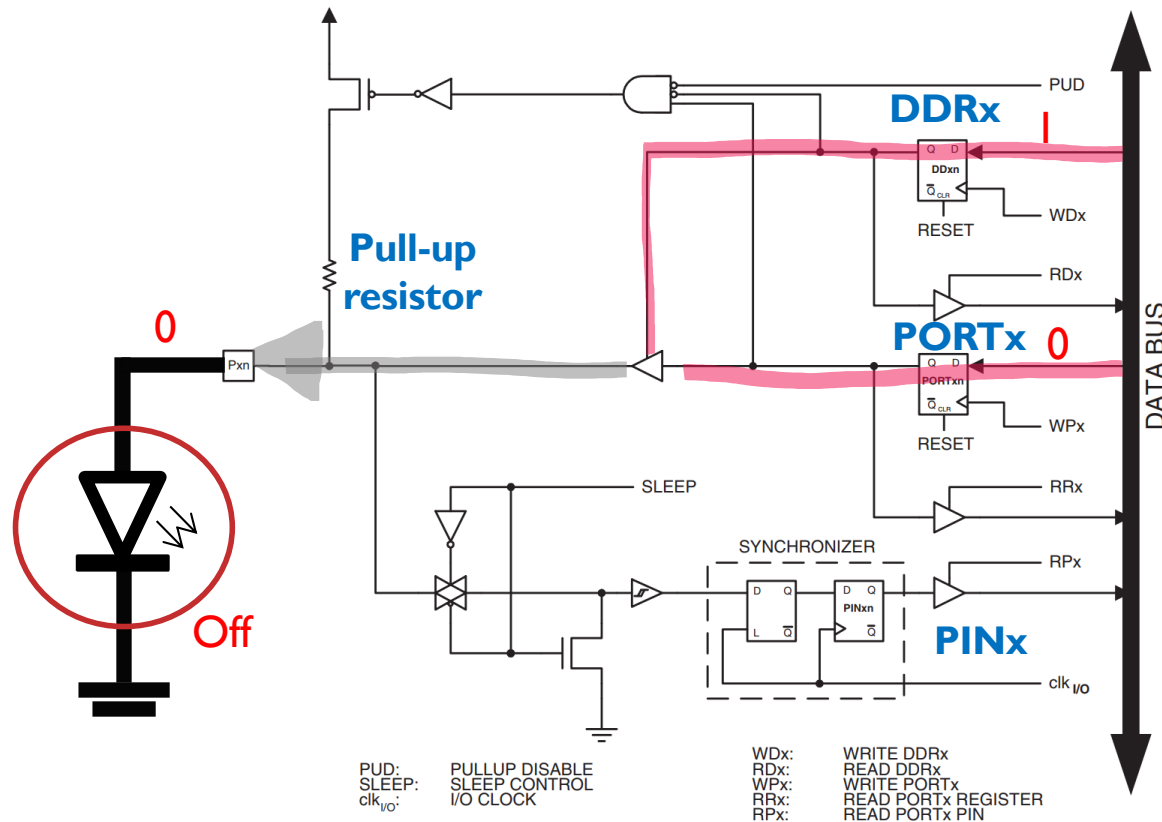
Figure 30. General Digital I/O⁽¹⁾



Note: 1. WPx, WDx, RRx, RPx, and RDx are common to all pins within the same port. clk_{I/O}, SLEEP, and PUD are common to all ports.

Configure for Output (2)

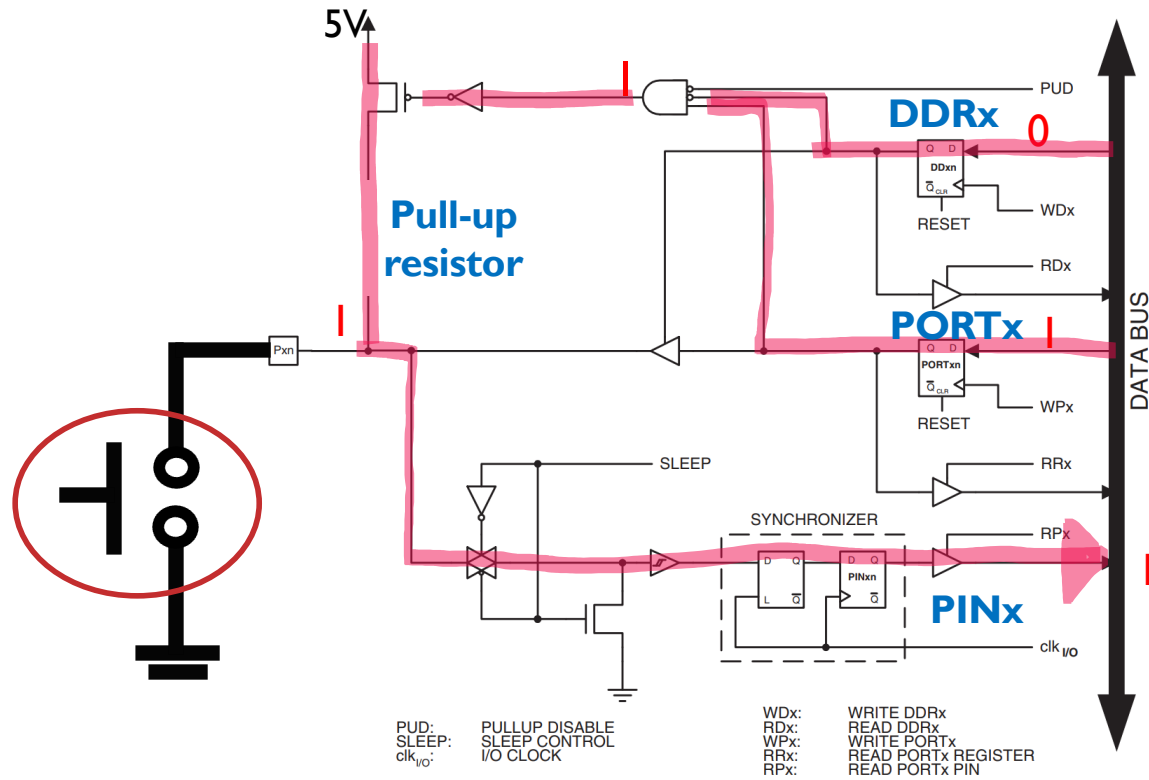
Figure 30. General Digital I/O⁽¹⁾



Note: 1. WPx, WDx, RRx, RPx, and RDx are common to all pins within the same port. clk_{I/O}, SLEEP, and PUD are common to all ports.

Configure for Input (I)

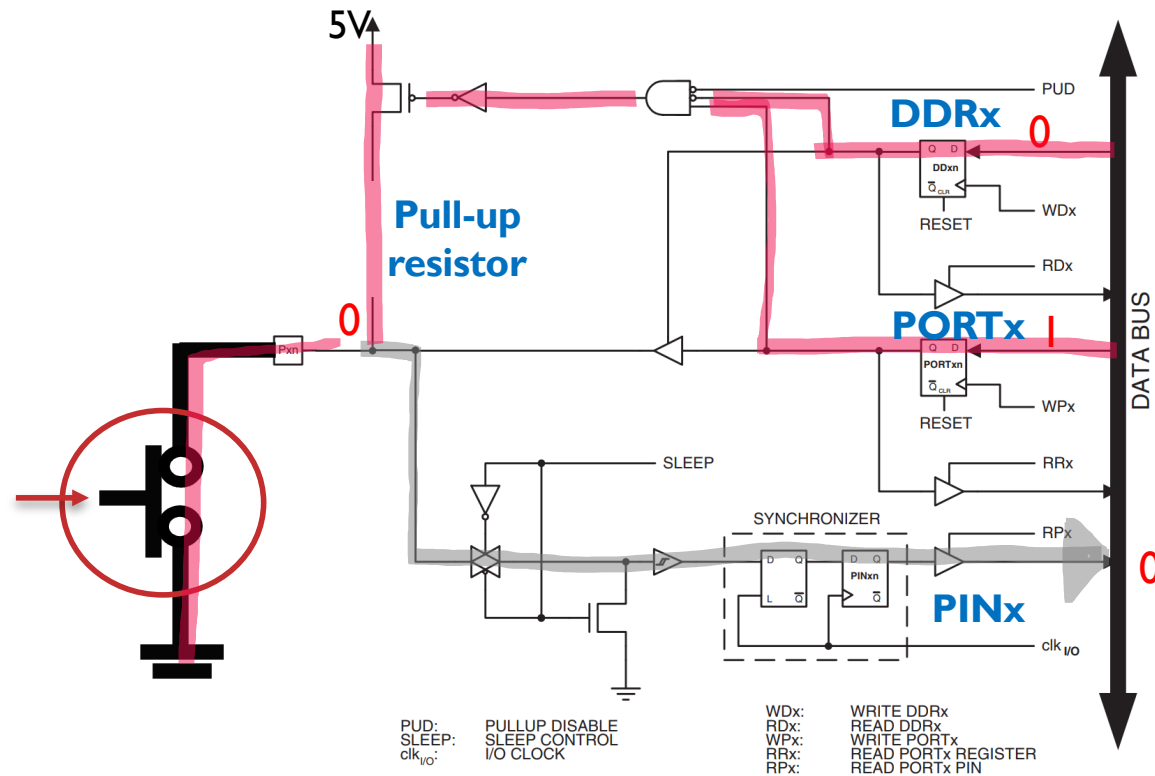
Figure 30. General Digital I/O⁽¹⁾



Note: 1. WPx, WDx, RRx, RPx, and RDx are common to all pins within the same port. clk_{I/O}, SLEEP, and PUD are common to all ports.

Configure for Input (2)

Figure 30. General Digital I/O⁽¹⁾



Note: 1. WPx, WDx, RRx, RPx, and RDx are common to all pins within the same port. clk_{I/O}, SLEEP, and PUD are common to all ports.

Switch Debouncing

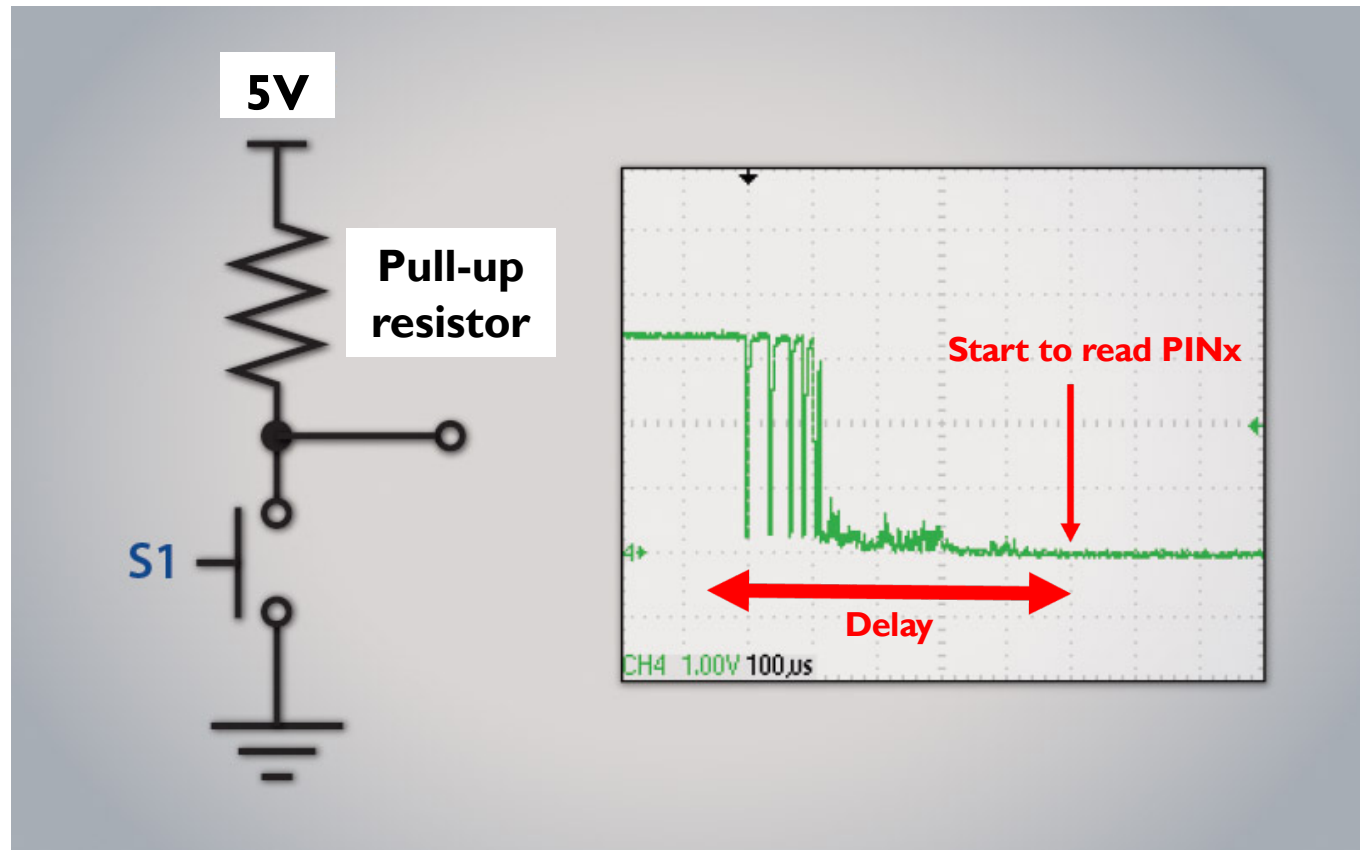


Image from google

C vs. Assembly (I)

- In C

```
DDRB = 0b11110000    // Set 7-4th bits as outputs
PORTB = 0b01100000    // Turn on LEDs connected to 5-6th
                        bits
```

- In Assembly

```
LDI      mpr, 0b11110000
OUT      DDRB, mpr      ; Set 7-4th bits as outputs
LDI      mpr, 0b01100000
OUT      PORTB, mpr     ; Turn on LEDs connected to 5-6th
                        bits
```

C vs. Assembly (2)

- In C

```
uint8_t mpr = PIND & 0b00110000; // Read and extract only 4-5th bits

If (mpr == 0b00100000) // Check if the right whisker is hit
{
    BotAction(); // Call BotAction
}
```

- In Assembly

IN	mpr, PIND	; Read and
ANDI	mpr, 0b00110000	; extract only 4-5 th bits
CPI	mpr, 0b00100000	; Check if right whisker is hit
BRNE	NEXT	; If no, go to NEXT
RCALL	BotAction	; if yes, call BotAction
NEXT:		



Check-off List

- Correctly initialize Ports for input and output.
- Correctly detect whisker inputs:
 - Left
 - Right
 - Both
- Correct Bumpbot behavior accordingly to which whisker is hit.
- Successfully translate the code file into a hex file.
- Successfully flash the code to the board.



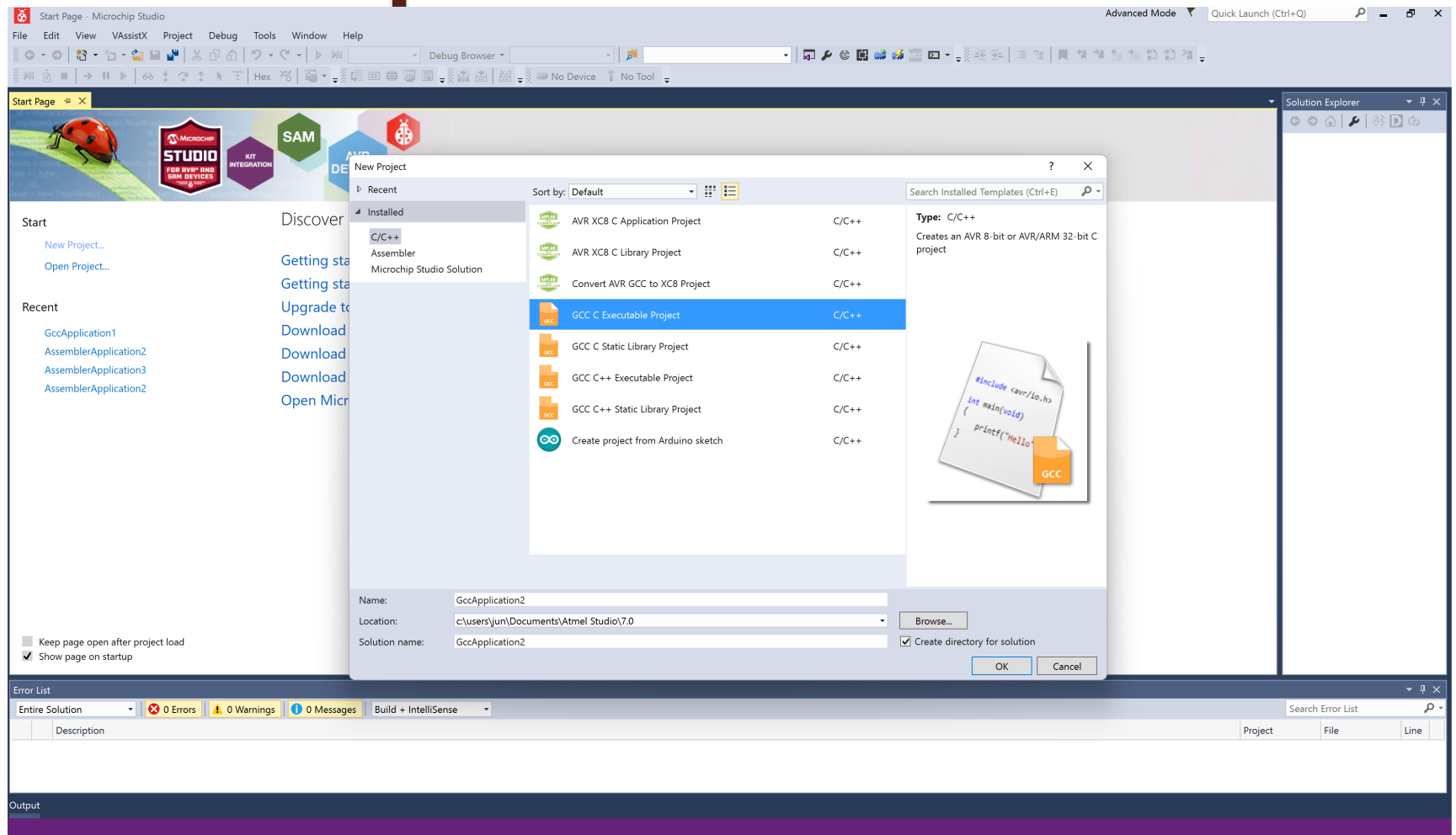
Announcements

- Lab I report is **required**:
 - Your report must use templates provided on the lab website.
 - The lab report should consist of the following (an example report is given on Canvas):
 - Introduction,
 - Design Document,
 - Program Overview,
 - Testing,
 - Study Questions,
 - Difficulties and Conclusion.
- Submit the following files by the due date shown on Canvas:
 - Lab I report
 - Source code

Questions?



C Compilation for Windows





C Compilation for Mac and Ubuntu Users

- Install avr-gcc toolchain
- Download Makefile in the Lab webpage
- Open the Makefile with a text editor and set `PRG` variable to the source code file name excluding file extension(.c).
 - e.g., `PRG = DanceBot`